

```
// Step 3. Return the reversed string
return newString; // " function reverseString(str) {
// Step 1. Create an empty string that will host the new created string
var newString = "";
// Step 2. Create the FOR loop
/* The starting point of the loop will be (str.length - 1) which
corresponds to the last character of the string, "o" As long as i is
greater than or equals 0, the loop will go on We decrement i after each
iteration */
for (var i = str.length - 1; i >= 0; i--) {
    newString += str[i]; // or newString = newString + str[i]; }
/* Here hello's length equals 5
For each iteration: i = str.length - 1 and newString = newString + str[i]
    First iteration: i = 5 - 1 = 4, newString = "" + "o" = "o"
    Second iteration: i = 4 - 1 = 3, newString = "o" + "l" = "ol"
    Third iteration: i = 3 - 1 = 2, newString = "ol" + "l" = "oll"
    Fourth iteration: i = 2 - 1 = 1, newString = "oll" + "e" = "olle"
    Fifth iteration: i = 1 - 1 = 0, newString = "olle" + "h" = "olleh"
End of the FOR Loop*/
// Step 3. Return the reversed string
return newString; // "yohA" }
reverseString('Ahoy'); " }
reverseString('Ahoy');
```

iv) Reversal of strings with recursion

For this solution, two methods are to be used: the `String.prototype.substr()` method and the `String.prototype.charAt()` method.

- The `substr()` method returns the characters back in a string beginning at the specified location through the number of characters specified.

```
"hello".substr(1); // "ello"
```

- The `charAt()` method returns the specified character from a string.
Note: while playing with recursion note that the method will incredibly be slow and the size of the stack would be of major concern.

```
function reverseString(str) {
    if (str === "") // This is the terminal case that will
end the recursion
    return "";
    else
```